SAST-DAST Correlation Tool: Hackathon Build Plan

A practical guide to building the MVP: how to build each step, which tools to use, where to take help, and how to present it.

1. The idea in plain words

Two kinds of scanners look at an app from different sides. **SAST** (Semgrep) reads the source code and says: this line looks dangerous. **DAST** (OWASP ZAP) attacks the running app and says: this URL really breaks. Each alone is noisy. Your tool joins them. If both point at the same problem, it is **Confirmed**, goes to the top of the list, and gets a plain-English fix.

Everything depends on one trick: the **route map**. It is the bridge between a URL (what ZAP sees) and a file and line number (what Semgrep sees). Build that bridge well and the rest is straightforward.

2. MVP scope: build this, skip that

Must have (this is your demo):

- Run Semgrep and ZAP against one app and save raw results.
- Convert both into one common finding format with CWE ids.
- Route map for Flask first, Express (Juice Shop) second.
- Correlation into Confirmed / Code only / Runtime only.
- Risk score and sorted list.
- Fix suggestion for Confirmed issues.
- A simple dashboard with an issue detail view.

Skip for the hackathon: user login for your own dashboard, CI/CD integration, machine learning, more languages, perfect accuracy, scanning anything that is not your own local test apps.

Golden rule: get a thin end-to-end version working on the Flask app first (a walking skeleton), then improve each stage. Do not polish step 2 before step 8 exists.

3. Recommended stack (all free, Python-first)

Part	Choice	Why
Core language	Python 3.11	Whole team can read it; the ast module is built in
SAST	Semgrep CLI with JSON output	One command, rich output, CWE metadata included
DAST	OWASP ZAP in Docker, driven by its API (python package zaproxy)	Scriptable spider, active scan and alerts
Targets	Docker containers for the Flask app and Juice Shop	Repeatable, easy reset between runs
API	FastAPI	Automatic interactive docs at /docs, which is your API demo for free
Storage	SQLite	Zero setup, enough for scan history
Dashboard	Streamlit (or plain HTML with HTMX)	Fastest way to a good-looking table and detail view
Flask route mapping	Python ast module, or Flask url_map	Exact and simple
Express route mapping	Small Node script (TypeScript Compiler API or Babel parser)	Needed for JS/TS source
Fix text	Template library, optionally enhanced by an LLM call	Reliable offline fallback
Packaging	docker-compose	One command to start everything for judges

4. Project layout and shared data model

```
sastdast/
  scanners/      semgrep_runner.py  zap_runner.py
  adapters/      base.py  flask_adapter.py  express_adapter.py
  core/          models.py  normalize.py  correlate.py  score.py  fixes.py
  api/           main.py            (FastAPI)
  dashboard/     app.py             (Streamlit)
  data/          raw scan JSON + scans.db (SQLite)
  docker-compose.yml
```

Define the data model **first** and share it with the whole team, so everyone builds against the same shapes:

- **Finding**: id, source (SAST or DAST), title, cwe (integer), severity (Low, Medium, High, Critical), rule_id, file, line_start, line_end, snippet, url, method, parameter, payload, evidence, raw (original JSON).
- **Route**: method, path_pattern, path_regex, file, handler_name, line_start, line_end.
- **Issue**: id, status (confirmed, code_only, runtime_only), cwe, severity, risk_score, sast_findings, dast_findings, route, fix_suggestion.

Use pydantic models so the same classes serve the API and the database layer.

The **adapter interface** is the key design idea from your workflow. Every language adapter exposes one function: discover_routes(source_dir) returns a list of Route. The core engine never knows which language it is dealing with.

5. Step-by-step build guide

Step 1. Select and configure the application

Goal: one config object that drives the whole scan.

How to build:

- Create a scan_config with source_dir, target_url, language, framework, and the Semgrep rule packs to use.
- Keep it as a YAML file or a form in the dashboard. Presets for flask-demo and juice-shop are enough.
- Choose the adapter from the language field (python selects the Flask adapter, javascript selects the Express adapter).
- Start both targets with docker-compose. Use compose service names (for example http://juice-shop:3000) so ZAP inside Docker can reach them.

Help to take: ask an AI assistant to generate the docker-compose.yml for ZAP, Juice Shop (image bkimminich/juice-shop) and your Flask app. Check the Docker docs for networking.

Done when: one command starts both apps and your tool can print the loaded config.

Watch out: only ever scan apps you run yourself, locally. Never expose intentionally vulnerable apps to a public network.

Step 2. Scan the source code (SAST)

Goal: a list of code-level findings with file, line and snippet.

How to build:

- Run Semgrep as a subprocess and read its JSON: `semgrep scan --config p/python --config p/owasp-top-ten --json --output sast.json ./app`
- For Juice Shop use the javascript, typescript and nodejs rule packs.
- For each item in results, read check_id, path, start.line, end.line, extra.message, extra.severity, extra.lines and extra.metadata.cwe.
- The CWE arrives as text such as CWE-89: Improper Neutralization... Extract the number with a regular expression.
- Do not build your own parser for finding vulnerabilities. Semgrep does that. Your parser work is only for route discovery in step 3.

Help to take: the Semgrep docs for the JSON output format; an AI assistant to write the runner and the CWE-extraction function.

Done when: you can print a clean table of file, line, CWE and severity for the Flask app.

Watch out (Juice Shop): it contains deliberately vulnerable example snippets (for example under data/static/codefixes), test files and a large frontend. Exclude them with the --exclude flag, or the results will be mostly noise. Start with server-side code only.

Step 3. Map application routes

Goal: a list of Route records, each tied to a handler function and its line range.

Flask adapter, easiest path (Python ast):

- Parse each .py file with ast.parse.
- Walk the tree for FunctionDef nodes whose decorators look like `@app.route('/path', methods=['GET'])`, `@bp.route(...)`, `@app.get(...)` or `@app.post(...)`.
- Record path, methods, function name, node.lineno and node.end_lineno.
- Convert Flask path variables (written like `<int:id>`) into a regex fragment (for example `[^/]+`) so URLs can be matched later.

- **Shortcut for the hackathon:** if you can import the Flask app, loop over app.url_map.iter_rules() and use inspect.getsourcelines on each view function. It gives the same data with less code.

Express adapter (Juice Shop):

- Write a small Node script using the TypeScript Compiler API (or @babel/parser) that finds calls shaped like app.get('/rest/products/search', handler) and router.post(...).
- Handlers are often imported from another file, for example routes/search.ts. Follow the import to get the handler file. If that gets hard, use the file-name heuristic (the route name usually matches the file in routes/).
- Print the result as JSON and read it from Python. The adapter interface stays the same.
- **Fallback if time runs out:** write a Semgrep rule that matches calls of the form app.METHOD(PATH, ...) to extract routes, or hand-write a routes.json for the 8 to 10 endpoints you demo. A hand-written map is an acceptable stop-gap, as long as you say so honestly in your report.

Help to take: the Python ast documentation, the ts-morph or TypeScript Compiler API docs, and an AI assistant to write the visitor code. Paste in two or three real route definitions from your target and ask for code that extracts them.

Done when: given a URL like /rest/products/search, a function find_route(method, url) returns the file and line range.

Step 4. Test the running application (DAST)

Goal: a list of runtime findings with URL, method, parameter, payload and evidence.

How to build:

- Start ZAP in daemon mode in Docker with an API key (image ghcr.io/zaproxy/zaproxy:stable, run zap.sh -daemon with the api.key config and the API address options set so other containers may connect).
- With the python zaproxy client: zap.spider.scan(url), then zap.ajaxSpider.scan(url) (Juice Shop is an Angular single-page app, so the normal spider alone finds little), then zap.ascan.scan(url).
- Poll status until it reaches 100, then call zap.core.alerts(baseurl=url).
- Each alert gives you alert (name), risk, confidence, url, method, param, attack (the payload), evidence, cweid and solution. This is exactly what your finding format needs.
- Save the raw alert JSON to data/ right away.

Help to take: the ZAP API docs and the ZAP automation and Docker docs; an AI assistant to write the polling loop.

Done when: a full run against the Flask app produces alerts and you have a saved file you can replay.

Watch out:

- Active scans on Juice Shop can take very long. Limit scope with a ZAP context that covers only a few endpoints, or set a maximum duration.
- Many Juice Shop endpoints need a login. Log in once through its REST login endpoint, take the token, and add it as a header via ZAP (Replacer rule), or simply demo the unauthenticated endpoints.
- **Save every raw scan output.** If the live scan fails on stage, you can run the rest of the pipeline from saved files.

Step 5. Normalise findings

Goal: SAST and DAST results look identical to the rest of the system.

How to build:

- Write two small functions, from_semgrep(item) and from_zap(alert), both returning a Finding.
- Severity: map everything to Low, Medium, High, Critical. Semgrep uses INFO, WARNING, ERROR; ZAP uses Informational, Low, Medium, High. Keep one lookup table.
- CWE: take it from Semgrep metadata and from ZAP cweid. If missing, use a small fallback table from rule or alert name to CWE (SQL injection 89, cross-site scripting 79, path traversal 22, OS command injection 78, hardcoded credentials 798).
- Add a CWE alias table for near matches (for example 80 and 83 count as 79) so the correlation step does not miss real matches.
- Use a stable id, for example a hash of source, CWE, location and parameter.

Help to take: the MITRE CWE site for the ids; an AI assistant to draft the mapping tables, which you then verify by hand.

Done when: one list contains both kinds of findings and every item has a CWE and a severity.

Step 6. Correlate, merge and classify

Goal: the core of the project. See Section 6 for the algorithm in detail.

How to build:

- For each DAST finding, find its route from the URL and method using the route map.
- For each SAST finding, find its route by checking whether its file and line fall inside a handler's range.

- Match pairs that share the same route and the same CWE (after aliasing) and give each pair a confidence score.
- Merge matched pairs into one Issue that keeps all evidence from both sides. Unmatched SAST findings become Code only, unmatched DAST findings become Runtime only.
- Deduplicate: several ZAP alerts for the same URL, parameter and CWE become one finding; several Semgrep hits on the same line and CWE become one.

Help to take: this is the part to design together on a whiteboard. Use an AI assistant for the code, but first write 5 or 6 test cases by hand (a pair that must match, a pair that must not).

Done when: a seeded SQL injection in the Flask app shows up as one Confirmed issue with both code and runtime evidence.

Step 7. Prioritise and suggest fixes

Goal: sorted issues, each with a readable fix for Confirmed ones (requirement B-4).

How to build:

- Risk score: see Section 7.
- Fix suggestions, two layers:
  - **Layer 1, templates (reliable):** a dictionary keyed by CWE. Each entry has three parts: what is wrong, why it matters, how to fix it, plus a small before and after code example. Cover your top 6 or 7 CWEs (89, 79, 78, 22, 798, 601, 352).
  - **Layer 2, LLM (optional bonus):** send the vulnerable snippet, the payload and the evidence to an LLM API and ask for a short explanation and corrected code. Ask for JSON output with fixed fields. If the call fails, fall back to the template.
- Keep the wording simple: one sentence for the problem, one for the risk, three or four lines for the fix.

Help to take: OWASP Cheat Sheet Series is the best source for fix wording (for example the SQL Injection Prevention and XSS Prevention cheat sheets). An AI assistant can draft the templates; you check them against the cheat sheets.

Done when: the Confirmed SQL injection shows parameterised-query advice using the actual code snippet.

Step 8. Display the report

Goal: a developer-friendly view and an API.

How to build:

- FastAPI endpoints: POST /scans (start a scan), GET /scans (history), GET /scans/{id}/issues (sorted list), GET /issues/{id} (detail).
- Store scans and issues in SQLite so history survives restarts.
- Streamlit dashboard: a sorted table with status badges (green for Confirmed, amber for Code only, blue for Runtime only), filters by status and severity, and a detail panel showing CWE, severity, URL, parameter, payload, file, line, code snippet, response evidence and the fix.
- Add a summary bar at the top: totals per status, and the raw findings count versus the merged issues count. That number is a strong demo moment.
- Add an Export JSON or HTML button for the report.

Help to take: Streamlit docs and an AI assistant for layout code. Give it your Issue model and ask for the table and detail view.

Done when: a judge can open the dashboard, click the top issue and read the whole story on one screen.

6. The correlation algorithm (explained simply)

Think of it as matching two lists of clues about the same crime scene.

1. **Link both sides to a route.** DAST: take the URL path and method, match against each route's path_regex. SAST: find the route whose file matches the finding and whose line range contains (or is closest to) the finding line.
2. **Score each SAST and DAST pair on the same route:**
   - Same CWE (after alias table): +50
   - Same route: +30
   - The DAST parameter name appears in the SAST snippet or handler code: +20
3. **Decide:** score 80 or more is Confirmed. If the CWE is the same but the route link is weak, you may mark it Likely and treat it as Confirmed with lower confidence, or leave it separate. Keep it simple and state your rule in the report.
4. **Pick the best match:** each finding joins at most one issue, choosing the highest score.
5. **Leftovers:** unmatched SAST is Code only; unmatched DAST is Runtime only.

Known weak spots, which you can mention honestly: the vulnerable line may sit in a helper function called by the handler, not in the handler itself (fallback: match by file only); and some issues, like missing security headers, have no code location at all, so they are naturally Runtime only.

7. Risk score (simple and explainable)

Use a 0 to 100 score that you can explain in one sentence:

- Severity points: Low 10, Medium 25, High 40, Critical 55.
- Evidence bonus: Confirmed +30, Runtime only +15, Code only +0.
- Exposure bonus: reachable without login +10, otherwise +0.
- Cap at 100, sort descending.

The reasoning to show: an issue that is both visible in the code and exploitable at runtime is more urgent than either alone.

8. Time plan

Built for roughly 36 hours. Scale the blocks proportionally for 24 or 48 hours.

Hours	Work	Milestone
0 to 3	Agree on scope, data model and layout. Start Docker targets. Pick the Flask app and note 4 to 6 known vulnerabilities (the ground truth).	Everything runs locally; models.py committed
3 to 10	Semgrep runner, ZAP runner, normalisation, Flask route adapter	Raw results saved for the Flask app
10 to 16	Correlation, scoring, template fixes, minimal API	**Walking skeleton: Flask end to end**
16 to 22	Dashboard and detail view	Demo-able on Flask
22 to 30	Express adapter, Juice Shop config, tuning of excludes and ZAP scope	Juice Shop produces a believable report
30 to 33	LLM fix suggestions (optional), history, export, accuracy table	Polish features
33 to 36	Report, slides, demo rehearsal, saved fallback data	Submission ready

The rule: if any block slips, cut from the later blocks (LLM fixes first, then extra Juice Shop endpoints), never from the walking skeleton.

9. Suggested team split (4 people)

- **Person A, scanners:** Docker setup, Semgrep and ZAP runners, saved raw outputs.
- **Person B, adapters:** Flask route mapper, then Express route mapper.
- **Person C, core engine:** normalisation, correlation, scoring, fix templates, tests.
- **Person D, product:** FastAPI, SQLite, dashboard, report, slides and demo script.

With 2 or 3 people, merge A with B and C with D. Hold a short integration check every 4 to 6 hours with the saved JSON files as the contract between roles.

10. Testing and the demo

Ground truth table. Before you scan, list the vulnerabilities you know exist in the Flask app (for example SQL injection on one route, reflected XSS on another, command injection, path traversal, a hardcoded secret that is code only, and one missing-header issue that is runtime only). After the run, check which ones your tool classified correctly. This single table is your evidence that the tool works.

Metrics to show:

- Raw findings versus merged issues (deduplication ratio).
- How many known vulnerabilities were found, and how many were Confirmed.
- Same numbers for Juice Shop, with a few spot-checked examples.

Demo script (4 to 5 minutes):

1. One sentence of the problem: scanners disagree and developers drown in noise.
2. Start a scan from the dashboard on the Flask app.
3. Show the unified list. Open the top Confirmed issue: code line, payload, evidence, fix.
4. Show one Code only and one Runtime only example and explain why they differ.
5. Switch to Juice Shop and show the same engine with a different adapter.
6. Close with the design principle: one core engine, thin adapters per language.

Always have a backup: pre-recorded scan results loaded from data/ so the demo still works if ZAP or Docker misbehaves on stage.

11. Outline for your written report

Keep it to 3 or 4 pages, with a diagram:

1. **Problem:** SAST has false positives, DAST lacks code context, developers need one list.
2. **Approach:** the 8-step pipeline and the adapter design.

3. **Architecture diagram:** scanners, adapters, normaliser, correlator, scorer, API and dashboard.
4. **Key techniques:** route map as the bridge, CWE alias matching, scoring rule, risk formula.
5. **Results:** the ground truth table and metrics for both apps.
6. **Limitations (be honest):** matching quality depends on route mapping, helper functions, authentication, DAST coverage, and rule quality. Judges respect clear limits.
7. **Future work:** more frameworks (Django, Spring), auto-generated patches, CI integration, trend tracking.

12. Risks and fallbacks

Risk	Fallback
ZAP active scan too slow	Limit scope to chosen endpoints, set a time cap, use saved results
Single-page app not crawled	Use the AJAX spider, or seed ZAP with a list of known URLs
Express route extraction takes too long	Semgrep-based extraction, or a hand-written routes file for demo endpoints
Too many noisy Semgrep results	Exclude test, codefix and frontend folders; filter to ERROR and WARNING
Correlation misses real pairs	Widen the CWE alias table; add file-level matching as a fallback
LLM API unavailable	Template fixes (already built as layer 1)
Live demo fails	Load saved scan JSON from data/

13. Where to take help

Documentation to bookmark: Semgrep docs (CLI and JSON output), ZAP API and Docker docs, Python ast docs, FastAPI and Streamlit docs, OWASP Cheat Sheet Series, MITRE CWE list, Juice Shop companion guide.

AI assistant use that works well:

- Give it your data model and ask for one function at a time, with a test.
- Paste real sample output (a Semgrep result, a ZAP alert) and ask for the converter.

- Paste three real route definitions and ask for the extraction code.
- Ask it to review your correlation rules and suggest edge cases.
- Verify anything security-related against OWASP or MITRE, since generated mappings and fix advice can be subtly wrong.

Example prompt for step 6:

Here are my Finding and Route models (paste them). Write correlate(sast_findings, dast_findings, routes) that links findings by route and CWE, scores each pair (same CWE 50, same route 30, parameter appears in snippet 20), merges pairs scoring 80 or more into a Confirmed Issue, and returns leftovers as Code only or Runtime only. Include 5 pytest cases.

People: ask hackathon mentors, especially security or backend engineers, to sanity-check your correlation rule and your fix wording. A 10-minute review is worth hours.

14. Quick checklist

- [ ] Data model agreed and committed
- [ ] Both targets run in Docker
- [ ] Raw Semgrep and ZAP output saved for Flask
- [ ] Flask routes mapped
- [ ] Findings normalised with CWE and severity
- [ ] Correlation produces Confirmed, Code only, Runtime only
- [ ] Risk score and sorted list
- [ ] Template fixes for Confirmed issues
- [ ] API and dashboard working
- [ ] Juice Shop adapter and results
- [ ] Ground truth table and metrics
- [ ] Report, slides and fallback data ready